

* 상속 (Inheritance)

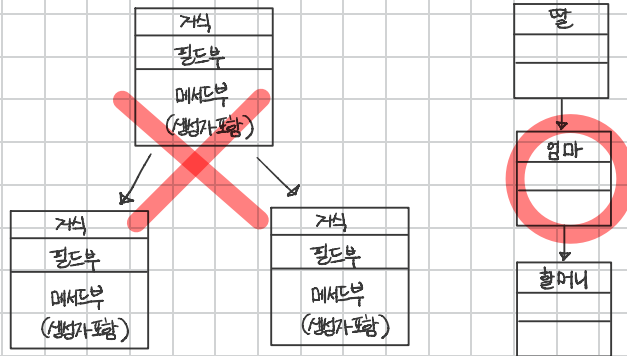
기존 클래스 (부모 클래스)가 가진 속성 (필드)과 기능 (메서드)을 새 클래스 (자식 클래스)가 물려받는 것.

Class & abstract Class

extends (자식 extends 부모 { })

↳ 엄마아빠 서우면 누굴지 고민되었어
상속 클래스가 복수이면 서로 상속하는 내용이
일때 자식클래스도 헷갈리니까 부모 클래스는
only one!

엄마의 엄마의 엄마 사르르 상속 가능.
엄마의 자식이 여러명이 것도 가능.



중녀 같은 느낌으로
모든 클래스의 조상적인
Object 클래스 존재

Interface Implements

(자식 Implements 인터페이스 1, 2, ... { })

↳ 배우하자면 선생님... 보통 생물학적 엄마는
하지만 선생님은 여러명이요, 선생님을 보고
배우므로 맞은 기능을 갖게 되지만 학생마다
개성에 따라 세부적으로 차이를 갖게 됨.

↳ **메서드 오버라이딩 ***

* 메서드 오버라이딩 조건

- 부모 메서드와
- 1) 같은 이름
 - 2) 같은 매개변수 정보
 - 3) 같은 반환형
 - 4) 부모 이상의 접근 제한 범위
 - 5) 부모 이하의 예외 선언

* 다형성 (Polymorphism)

한 객체 (참조변수)가 여러 형태를 가지는 능력

부모 타입의 참조변수로 자식 타입 인스턴스 참조 가능.

↳ 객체 배열, 매개변수 선언할 때는 부모타입으로 통쳐서 선언하고
말씀할 때는 자식 클래스 인스턴스를 가져와도 된다.

* 캡슐화 (Encapsulation)

객체의 속성 (데이터, 필드, 멤버변수)과 기능 (메서드)을
하나로 묶어 내부에서 스스로 관리하게 하는 것.

↳ 대표 원칙: 정보 은닉.

데이터 보호: 외부로부터 내부 데이터로의 직접 접근 방지.

⇒ 거점에서 캐릭터의 hp (필드)에 직접 접근하여
hp의 설정값을 100배로 만드는 등의 치트 행위 방지

⇒ 웹 사이트 등에서 사용자의 pw에 직접 접근하여
임의로 설정값을 바꾸지 못하게 할 수 있음.

무결성 유지: 객체가 자신의 데이터를 스스로 검증 및 관리하여
데이터 오염 방지.

유지보수성 향상: 내부구현이 바뀌어도 외부에 미치는 영향을
최소화.

* 추상화 (Abstraction)

프로그램에 필요한 공통점은 추출하고,

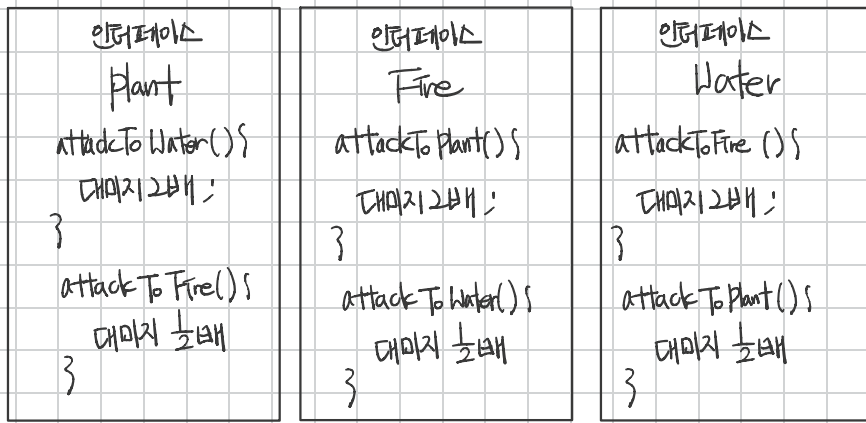
불필요한 특성은 제거하는 과정.

↳ **대여소 매장**

공통으로 필요한 특성: 매장 위치, 매장 규모, 직원수 등

필요 없는 특성: 매장 주변 풍경, 직원의 생김새 등

ex) 포켓몬



어떤 인터페이스인지 확인하는
용어로, 클래스 다이어그램의
행위를 고려하지 않았습니다.

```

@ Override // 모부기
public void attack() {
    System.out.println("플레기");
}

@ Override // 파이어
public void attack() {
    System.out.println("불꽃 세례");
}

@ Override // 꼬박기
public void attack() {
    System.out.println("물대포");
}
  
```

Pokemon p1 = new Pokemon();

Pokemon p2 = new 파이어();

파이어 객체가 부모 타입인 Pokemon으로

업캐스팅됨. 다형성 적용

자식 객체는 힙 메모리에 생성 시 부모의
멤버 공간을 항상 내포하고 있다.

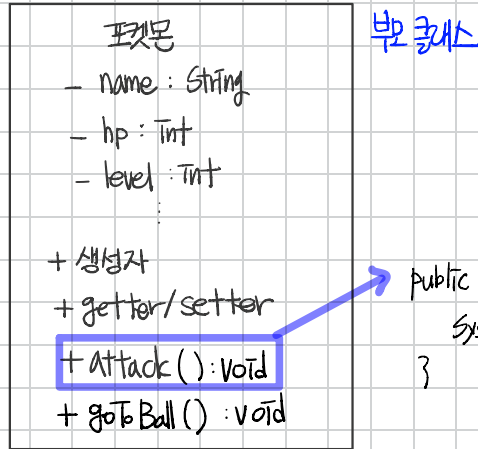
p1.attack(); // 결과: 뚱뚱박치기

정적 바인딩: 참조 타입과 인스턴스 타입 일치.

p2.attack(); // 결과: 불꽃 세례

동적 바인딩: 컴파일 단계에서는 참조 타입의 메서드라

다형성 적용. 연결(정적 바인딩)이지만, 실행 단계에서는
실제 힙 영역에 있는 인스턴스의 오버라이딩
규칙을 확인하고 부모의 메서드가 아닌, 자식의
오버라이딩 버전 메서드가 최종 호출되는 것.



```

public void attack() {
    System.out.println("뚱뚱박치기");
}
  
```

if (p2 instanceof 파이어) {

int temperature = ((파이어) p2).getTailFlameTemperature();

System.out.println(temperature);

} 다운캐스팅: 업캐스팅으로 인해 제한된 자식 고유의 기능을
사용하기 위해 자식 타입으로 강제 형변환.

ClassCastException 발생 방지를 위해 instanceof 사용.